

RUL Estimation of Turbofan Engines Using Change Point Detection Technique

M.L. Course 2024 / 2025

Napoli Fabian 838152

Summary

1. Introduction	1
2. Related Work	1
3. Methodology	2
4. Experiments and Results	4
5. Discussion	5
6. References.....	6

Abstract

This paper addresses the critical problem of Remaining Useful Life (RUL) estimation for turbofan jet engines. Traditional approaches, as seen in [1] and [2], rely on static labeling of the dataset to train machine learning models for RUL estimation. We propose a novel dynamic labeling approach based on change point detection in sensor data. Our strategy is to identify significant shifts in sensor patterns, using these change points for RUL labelling. Then machine learning algorithms were used to estimate the RUL using the new proposed approach. Experimental results demonstrate that applying machine learning algorithms to this new dynamic labeling strategy yields better performance compared to the previously proposed techniques.

1. Introduction

The estimation of the Remaining Useful Life (RUL) of jet engines is a critical task in predictive maintenance and safety management within the aerospace industry. Accurate RUL estimate enables timely maintenance, reduces operational costs, and prevents unexpected failures. Using the dataset provided by NASA, which simulates turbofan engine degradation through CMAPSS simulations, we face the challenge of estimating the target variable ourselves, as it is not explicitly included. The true RUL values are only available for the test set, at the end of each engine's lifecycle. [5]

2. Related Work

In the following case studies, two approaches have been applied to address this task:

- The first approach involves labeling the training set by assuming that the RUL decreases linearly from the maximum recorded cycle for each engine down to zero [2]. This method has shown limited effectiveness, as the predictive models trained with this assumption often result in poor accuracy.
- The second approach follows the same procedure but imposes a fixed RUL threshold (e.g., 120 or 130 cycles). While this improves results compared to the first method, it still relies on a strong assumption and restricts the models to predicting RUL values only from that threshold [1][4];

Our work proposes a dynamic labeling approach: an unsupervised technique is used to label the RUL by applying change point detection algorithm to dynamically annotate the dataset. This labeling then enables the use of regression techniques to use Machine Learning algorithm to estimate the RUL.

3. Methodology

Dataset

The dataset includes multivariate time series, each representing the lifecycle of a different engine with unknown initial conditions. In the training set, engines run normally before gradually degrading to failure. In the test set, engines are observed only up to a point before failure, with ground-truth RUL values provided separately. For this study, we focus on the FD001 dataset. [5]

Removed Features

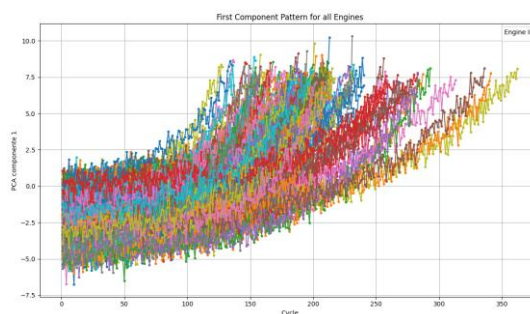
The first step is to understand how the feature data varies. By plotting the data against cycles, we observe that some features have constant, and therefore uninformative, values. These features will be removed:

['sensor_1', 'sensor_5', 'sensor_6', 'sensor_9', 'sensor_10', 'sensor_16', 'sensor_18', 'sensor_19', 'op_setting_3', 'op_setting_2', 'op_setting_1']

On training the models, Engine_id and cycle were removed to avoid memorization problems.

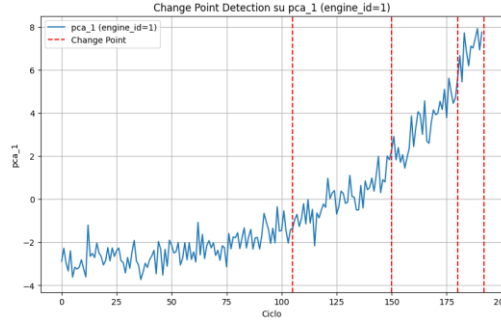
The remain features were scaled using standardScaler() provided by the library scikit-learn.

Features Fusion



To implement our strategy, which aims to identify the point where the data begins to change significantly, indicating the onset of a failure we use Principal Component Analysis (PCA): we utilize the first component obtained to identify the failure point.

Change Point Detection



Now, for each engine in the training set, we identify the change point using the newly derived feature. To achieve this, we use the ruptures library's models: We treat the first principal component as a time-series signal using the cycles and apply the models provided by ruptures to detect the change point. Specifically, we use this parameters:

- `rpt.Binseg(model="l2");`
- `l2 = squared error loss;`
- `n_bkps=3`, number of change point to find;

Data Labelling

The first step is to label the training set with the standard approach based on the maximal cycle recorded in the set and then linear decrease until total failure:

Algorithm 1: RUL with Linear Degradation

Data: Train Data D

```

1 while engine in  $D(\text{engines})$  do
2   maxCycle = engine.cycle.max();
3   RULS = [];
4   for cycle in engine.cycle do
5     RUL = maxCycle - cycle;
6     RULS.add(RUL);
7   end
8   engine.RUL = RULS;
9 end

```

Now using the change points identified, we limit the maximum RUL value for each engine to the corresponding change point:

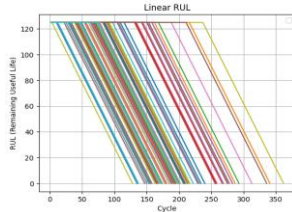
Algorithm 2: Dinamic PieceWise RUL Estimation

Data: Train Data D

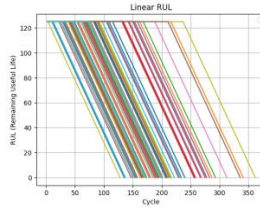
```

1 while engine in  $D(\text{engines})$  do
2   changePoint = engine.changePoint;
3   RULS = engine.RUL;
4   for RUL in RULS do
5     if RUL > changePoint then
6       RUL = changePoint
7     end
8   end
9 end

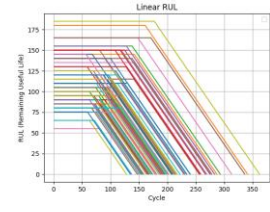
```



Linear Approach



RUL "clip" approach



Proposed approach

For testing the models we create a new test set in which each row corresponds to the last test cycle for each engine, labeled with the corresponding engine's RUL in the RUL's file.

Model Selection

The following models use parameters that have already been optimized either through grid search or by repeatedly training with different parameter settings.

Model	Parameters																											
Random Forest	random_state=42, n_estimators=300																											
KNN	n_neighbors=17																											
Feed-Forward Neural Network (NN)	<table><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr><tr><td>dense_4 (Dense)</td><td>(None, 256)</td><td>1,369</td></tr><tr><td>batch_normalization_2 (BatchNormalization)</td><td>(None, 256)</td><td>1,024</td></tr><tr><td>dropout_2 (Dropout)</td><td>(None, 256)</td><td>0</td></tr><tr><td>dense_5 (Dense)</td><td>(None, 128)</td><td>32,896</td></tr><tr><td>batch_normalization_3 (BatchNormalization)</td><td>(None, 128)</td><td>512</td></tr><tr><td>dropout_3 (Dropout)</td><td>(None, 128)</td><td>0</td></tr><tr><td>dense_6 (Dense)</td><td>(None, 64)</td><td>8,256</td></tr><tr><td>dense_7 (Dense)</td><td>(None, 1)</td><td>65</td></tr></table> EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True), epochs=200, batch_size=32, (optimizer=Adam(learning_rate=0.001), loss='mean_squared_error', metrics=['mse'])	Layer (type)	Output Shape	Param #	dense_4 (Dense)	(None, 256)	1,369	batch_normalization_2 (BatchNormalization)	(None, 256)	1,024	dropout_2 (Dropout)	(None, 256)	0	dense_5 (Dense)	(None, 128)	32,896	batch_normalization_3 (BatchNormalization)	(None, 128)	512	dropout_3 (Dropout)	(None, 128)	0	dense_6 (Dense)	(None, 64)	8,256	dense_7 (Dense)	(None, 1)	65
Layer (type)	Output Shape	Param #																										
dense_4 (Dense)	(None, 256)	1,369																										
batch_normalization_2 (BatchNormalization)	(None, 256)	1,024																										
dropout_2 (Dropout)	(None, 256)	0																										
dense_5 (Dense)	(None, 128)	32,896																										
batch_normalization_3 (BatchNormalization)	(None, 128)	512																										
dropout_3 (Dropout)	(None, 128)	0																										
dense_6 (Dense)	(None, 64)	8,256																										
dense_7 (Dense)	(None, 1)	65																										
LSTM	<table><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr><tr><td>lstm_62 (LSTM)</td><td>(None, 64)</td><td>19,968</td></tr><tr><td>dense_70 (Dense)</td><td>(None, 32)</td><td>2,080</td></tr><tr><td>batch_normalization_30 (BatchNormalization)</td><td>(None, 32)</td><td>128</td></tr><tr><td>dense_71 (Dense)</td><td>(None, 1)</td><td>33</td></tr></table> Timesteps = 30, epoch = 10, batch_size = 32, Adam l = 0.001, loss = mean_squared_error	Layer (type)	Output Shape	Param #	lstm_62 (LSTM)	(None, 64)	19,968	dense_70 (Dense)	(None, 32)	2,080	batch_normalization_30 (BatchNormalization)	(None, 32)	128	dense_71 (Dense)	(None, 1)	33												
Layer (type)	Output Shape	Param #																										
lstm_62 (LSTM)	(None, 64)	19,968																										
dense_70 (Dense)	(None, 32)	2,080																										
batch_normalization_30 (BatchNormalization)	(None, 32)	128																										
dense_71 (Dense)	(None, 1)	33																										

4. Experiments and Results

The new dataset with label was divided into a training and validation subset, with 90% of the data used for learning and the remaining 10% reserved for validation. Principal Component Analysis (PCA) is then applied, retaining the components that explain 99% of the total variance, resulting in 13 components out of the original 15.

Metrics

To compare the performance of the models performance we will use this metrics in order to have a complete and detailed evaluation:

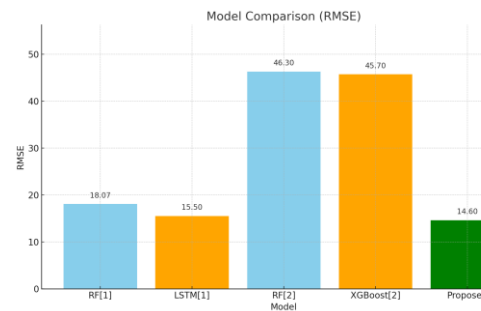
- Root mean-squared error (RMSE)
- Mean absolute error (MAE)
- Root relative squared error (RRSE)
- Relative absolute error (RAE)
- Correlation coefficient (CC).

Results

Models	RMSE	MAE	RRSE	RAE	CC	R2
RF	19.43	13.3	0.43	0.36	0.9	0.81
KNN	18.70	14.0	0.45	0.38	0.89	0.79
NN	17.70	13.2	0.42	0.36	0.90	0.81
LSTM	14.63	11.0	0.35	0.29	0.93	0.87

Baseline comparison

Model	RMSE
RF[1]	18.07
LSTM[1]	15.5
RF[2]	46.3
XGBoost[2]	45.7
Proposed	14.6



5. Discussion

As we can see, the selected models perform better than those proposed in the other case study, indicating that our strategy provides a starting point for more advanced methodologies based on pattern recognition and change point detection. However, despite the performance improvements, it is important to note that:

- The test set remains relatively small, so the estimated results may be optimistic;
- The sensor data is affected by noise. On developing simple smoothing techniques like moving averages lead to worse performance on training, suggesting the need for more sophisticated denoising methods;
- The degradation pattern of the RUL is a piecewise function, so basically is an approximation of the real RUL degradation pattern. A exponential approach could improve the performance;

Therefore, these observations highlight opportunities to enhance the approach by extending the project and addressing the outlined challenges.

6. References

- [1] Parisi, F. (n.d.). *NASA Turbofan Jet Engine – Kaggle Notebook*. Retrieved from <https://www.kaggle.com/code/fernandoparisi/nasa-turbofan-jet-engine>
- [2] Dighisci. (n.d.). *Prediction of NASA Turbofan Jet Engine RUL: OLS, SciKit-Learn & LSTM – Our Blogs*. Retrieved from <https://dighisci.com/prediction-of-nasa-turbofan-jet-engine-rul-ols-sci-kit-learn-lstm-our-blogs/>
- [3] Truong, C., Oudre, L., & Vayatis, N. (n.d.). *ruptures: change point detection in Python*. Retrieved from <https://pypi.org/project/ruptures/>
- [4] Malhotra, R. (n.d.). *Predictive maintenance on NASA's Turbofan Engine Degradation dataset (CMAPSS)*. Medium. <https://medium.com/@rohitmalhotra2604>
- [5] Behrad3d. (n.d.). *NASA CMAPSS: NASA turbofan jet engine data set [dataset]*. Kaggle. <https://www.kaggle.com/datasets/behrad3d/nasa-cmaps>